

作业 13

——精确求最近点对——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

给出三维空间中的 n 个点，请求出其中最近的两个点之间的距离。

输出保留两位小数!!!!

输入

第一行一个正整数 $n \leq 10^5$ 表示点的数量。接下来 n 行，每行三个小于 2^{30} 的非负整数，表示一个点的坐标（输入中不同数据点可能有相同坐标）。

输出

输出一个实数（保留两位小数!），表示最近点对的距离。

样例输入

```
5
677935270 699854128 417039459
248806312 20966623 311648820
675738077 1072105920 609689069
151071486 945372189 989472438
568196392 95711909 573356772
```

样例输出

```
419154024.84
```

2. 基于四分树的 WSPD

WSPD 是指 Well-Separated Pair Decomposition，即良分离对分解。它是一种将点集分解成一系列良分离对的技术，常用于解决最近点对问题等几何问题。形象地说，WSPD 将点集分解成若干对，每对中的两个子集之间的距离远大于它们内部的距离，从而使得在这些子集中寻找最近点对变得更容易。

$$\forall i, A_i, B_i \subseteq P, \max(\text{diam}(A_i), \text{diam}(B_i)) \leq \epsilon * \text{dist}(A_i, B_i)$$

WSPD 的核心就是找到一些划分地方式，使得每个集合中的点之间的距离较小而与其他点集的距离较大。常见的实现方法（我唯一可能会的）就是四叉树。具体操作是：

- 把整个点集放入一个矩形边框；
- 递归将这个矩形边框均匀分成若干小块，直到这里面的点很少或者格子足够小。
- 两两检查这些盒子：
 - ▶ 如果两盒子距离远大于它们的直径，就生成一个“良好分隔对”（两个盒子各自代表一个团）；
 - ▶ 否则，把较大的盒子继续拆分成子盒子，再与新盒子比较。

接下来我们一点点看看代码如何实现。

3. 代码实现

3.1. 准备工作

```
using ll = long long;
struct Points{
    ll x[3]; int id; //点的编号
};
struct Node{
    Points p; ll minn[3], maxn[3]; //当前节点的边界
    int indexLeft, indexRight; //左右子树的编号
};
```

3.2. 经过 D 老师的教导

我们会用到 `nth_element` 函数来找到中位数点。这个函数的作用是将序列中第 `n` 小的元素放到它排序后应该在的位置上，并保证：

- 该元素左边的所有元素都 $\leq$ 它；
- 该元素右边的所有元素都 $\geq$ 它。

但左右两侧内部不一定有序。

```
std::nth_element(_RandomAccessIterator __first, _RandomAccessIterator __nth,
    _RandomAccessIterator __last, _Compare __comp)
```

其中 `first` 和 `last` 是输入范围的迭代器，`nth` 是指向要放置第 `n` 小元素位置的迭代器。使用 `nth_element` 可以在 $O(n)$ 的平均时间复杂度内找到第 `n` 小的元素。因此，我们编写 `cmp` 函数：（全局变量也粘出来了）

```
Points pts[100005];
Points temp_pts[100005];
Node trees[100005];
int cur_dim; //当前维度
ll ans = LLONG_MAX; //全局最小距离
bool cmp(const Points &a, const Points &b) {
    if (a.x[cur_dim] != b.x[cur_dim])
        return a.x[cur_dim] < b.x[cur_dim];
    return a.id < b.id; // 坐标相同时按id排序，保证确定性
}
```

3.3. 更新节点的包围盒

每个节点的包围盒必须包含自身和所有子节点的点。递归构建完左右子树后调用此函数，自底向上更新所有节点的包围盒。

```
void upDate(int tr){
    int l = trees[tr].indexLeft, r = trees[tr].indexRight;
    for(int i = 0 ; i < 3; i ++){ // 初始包围盒为当前节点自身的坐标
        trees[tr].minn[i] = trees[tr].maxn[i] = trees[tr].p.x[i];
        if(l != 0){
            trees[tr].minn[i] = min(trees[tr].minn[i], trees[l].minn[i]);
            trees[tr].maxn[i] = max(trees[tr].maxn[i], trees[l].maxn[i]);
        }
        if(r != 0) {
            trees[tr].minn[i] = min(trees[tr].minn[i], trees[r].minn[i]);
            trees[tr].maxn[i] = max(trees[tr].maxn[i], trees[r].maxn[i]);
        }
    }
}
```

3.4. 建树

每个维度交替进行，递归构建两边的子树。

```
int build(int l, int r, int d) {
    if (l > r) return 0; // 空区间返回0 (空节点)

    int mid = (l + r) >> 1; // 选择中间位置作为当前节点 相当于除以2向下取整。
    cur_dim = d;           // 设置当前划分维度

    // 将第mid小的元素放到mid位置，左边≤它，右边≥它 (O(n)时间)
    nth_element(temp_pts + l, temp_pts + mid, temp_pts + r + 1, cmp);

    trees[mid].p = temp_pts[mid]; // 存储当前节点的点
    // 递归构建左右子树，维度切换为(d+1)%3 (三维循环)
    trees[mid].indexLeft = build(l, mid - 1, (d + 1) % 3);
    trees[mid].indexRight = build(mid + 1, r, (d + 1) % 3);
    upDate(mid); // 更新当前节点的包围盒
    return mid; // 返回当前节点的索引
}
```

3.5. 剪枝

剪枝原理：这个函数返回的是点到子树中所有点的最短距离的下界。也就是说，子树中任何点到 pt 的距离平方都 $\geq$ 这个值。如果这个下界已经大于当前找到的最小距离 ans，那么这个子树中不可能有更近的点，可以直接跳过，不用递归搜索。

```
// 计算两点之间的欧几里得距离平方（避免开根号，提高速度和精度）
inline ll dist_sq(const Point &a, const Point &b) {
    ll res = 0;
    for (int i = 0; i < 3; i++) {
        ll d = a.x[i] - b.x[i];
        res += d * d;
    }
    return res;
}

// 计算点pt到节点rt的包围盒的最短距离平方（用于剪枝）
inline ll guess_min_sq(int rt, const Points &pt) {
    ll res = 0;
    for (int i = 0; i < 3; i++) {
        if (pt.x[i] < trees[rt].minn[i]) {
            // 点在包围盒左侧，该维度距离为minn[i]-pt.x[i]
            ll d = trees[rt].minn[i] - pt.x[i];
            res += d * d;
        } else if (pt.x[i] > trees[rt].maxn[i]) {
            // 点在包围盒右侧，该维度距离为pt.x[i]-maxn[i]
            ll d = pt.x[i] - trees[rt].maxn[i];
            res += d * d;
        }
        // 点在包围盒内部，该维度距离为0
    }
    return res;
}
```

3.6. 处理询问

```
void query(int rt, const Points &pt) {
```

```

if (!rt) return; // 空节点直接返回
// 计算当前节点的点与pt的距离平方，排除点自身
if (trees[rt].p.id != pt.id)
    ans = min(ans, dist_sq(trees[rt].p, pt));
int l = trees[rt].indexLeft, r = trees[rt].indexRight;
// 计算左右子树的包围盒到pt的最短距离平方
ll dl = l ? guess_min_sq(l, pt) : LLONG_MAX;
ll dr = r ? guess_min_sq(r, pt) : LLONG_MAX;
// 优先搜索距离更近的子树（更快找到更小的ans，增强剪枝效果）
if (dl < dr) {
    if (dl < ans) query(l, pt); // 只有下界小于当前最小值才搜索
    if (dr < ans) query(r, pt);
} else {
    if (dr < ans) query(r, pt);
    if (dl < ans) query(l, pt);
}
}
}

```

3.7. 主函数

```

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    // 边界情况：少于2个点时距离为0
    if (n < 2) {
        printf("0.00\n");
        return 0;
    }
    // 读取所有点
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%lld", &pts[i].x[j]);
        }
        pts[i].id = i; // 给每个点分配唯一id
        temp_pts[i] = pts[i]; // 复制到临时数组用于构建KD-Tree
    }
    int root = build(0, n - 1, 0);
    // 对每个原始点查询其最近邻
    for (int i = 0; i < n; i++) {
        query(root, pts[i]);
    }
    // 开根号得到实际距离，保留两位小数输出
    printf("%.2f\n", sqrt((double)ans));
    return 0;
}

```